

AI Assisted Coding Lab ASS-11.1

Name : G.Sanjay

Roll no-2303A510B9

B_13

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty

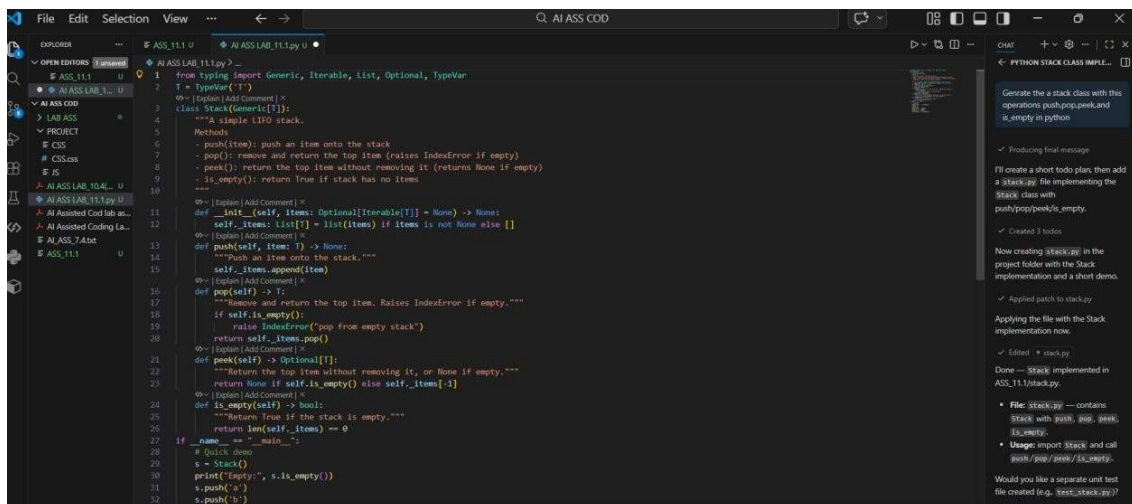
methods. Sample Input Code: class Stack:

pass

Expected Output:

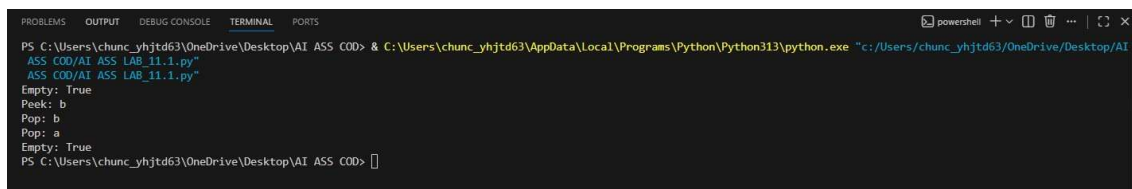
- A functional stack implementation with all required methods and docstrings.

PROMPT: Genrate the a stack class with this operations push,pop,peek,and is_empty in python



```
1 from typing import Generic, Iterable, List, Optional, TypeVar
2 T = TypeVar('T')
3 class Stack(Generic[T]):
4     """A simple LIFO stack.
5     Methods
6     - push(item): push an item onto the stack
7     - pop(): remove and return the top item (raises IndexError if empty)
8     - peek(): return the top item without removing it (returns None if empty)
9     - is_empty(): return True if stack has no items
10    """
11    def __init__(self, items: Optional[Iterable[T]] = None) -> None:
12        self._items = List[T] = list(items) if items is not None else []
13    def push(self, item: T) -> None:
14        """Push an item onto the stack."""
15        self._items.append(item)
16    def pop(self) -> T:
17        """Remove and return the top item. Raises IndexError if empty."""
18        if self.is_empty():
19            raise IndexError("pop from empty stack")
20        return self._items.pop()
21    def peek(self) -> Optional[T]:
22        """Return the top item without removing it, or None if empty."""
23        return None if self.is_empty() else self._items[-1]
24    def is_empty(self) -> bool:
25        """Return True if the stack is empty."""
26        return len(self._items) == 0
27    if __name__ == "__main__":
28        # Quick demo
29        s = Stack()
30        print("Empty:", s.is_empty())
31        s.push('a')
32        s.push('b')
```

OUTPUT:



```
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS COD/AI ASS LAB 11.1.py"
Empty: True
Push: b
Push: a
Pop: a
Pop: b
Empty: True
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD>
```

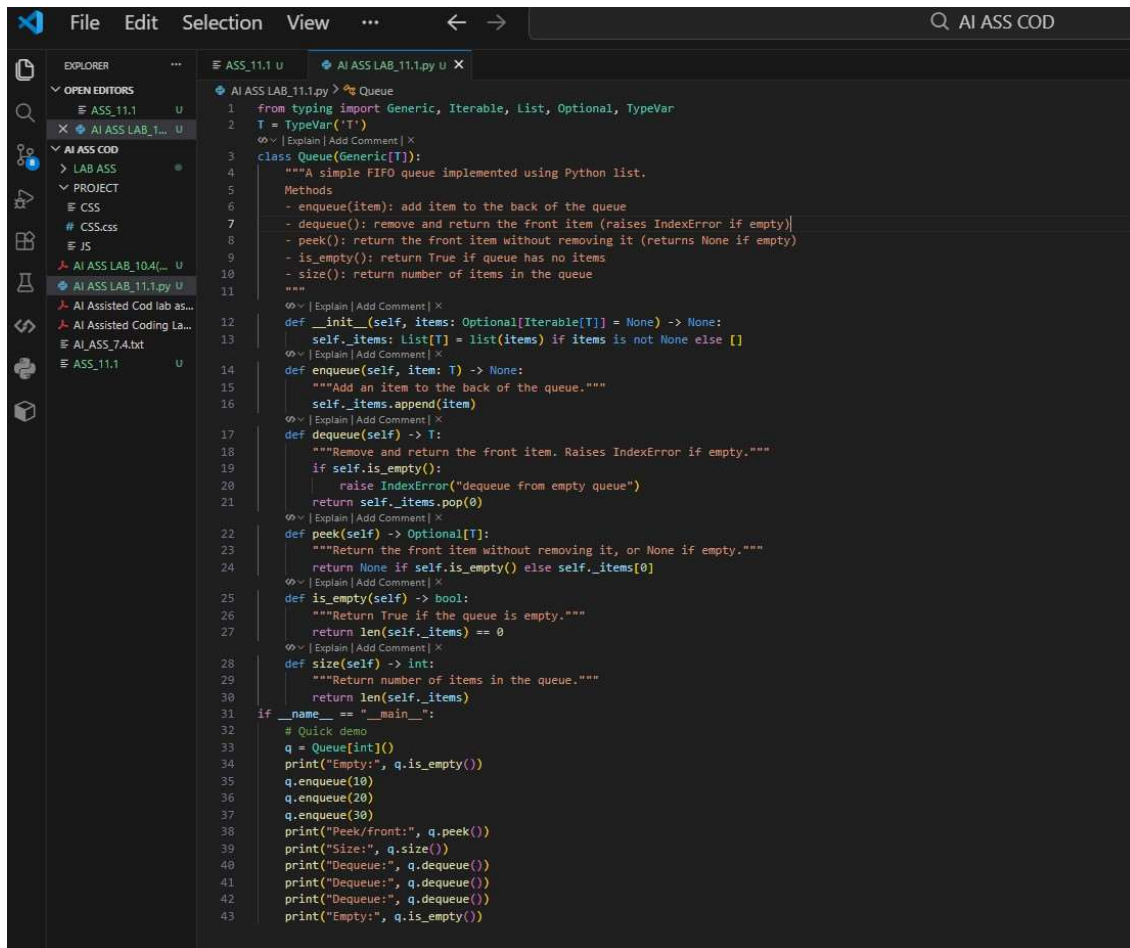
Task Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

Sample Input Code: class

Queue:

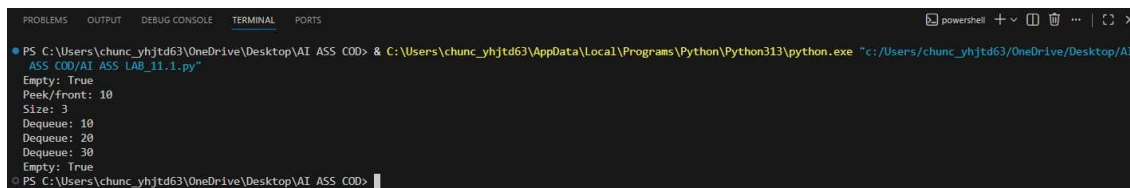
PROMPT: Generate queue using python list and perform its operations



The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'AI ASS COD' with subfolders 'LAB ASS' and 'PROJECT'. The code editor shows a file named 'AI ASS LAB_11.1.py' with the following code:

```
1 from typing import Generic, Iterable, List, Optional, TypeVar
2 T = TypeVar('T')
3 class Queue(Generic[T]):
4     """A simple FIFO queue implemented using Python list.
5     Methods
6     - enqueue(item): add item to the back of the queue
7     - dequeue(): remove and return the front item (raises IndexError if empty)
8     - peek(): return the front item without removing it (returns None if empty)
9     - is_empty(): return True if queue has no items
10    - size(): return number of items in the queue
11    """
12    def __init__(self, items: Optional[Iterable[T]] = None) -> None:
13        self._items: List[T] = list(items) if items is not None else []
14    def enqueue(self, item: T) -> None:
15        """Add an item to the back of the queue."""
16        self._items.append(item)
17    def dequeue(self) -> T:
18        """Remove and return the front item. Raises IndexError if empty."""
19        if self.is_empty():
20            raise IndexError("dequeue from empty queue")
21        return self._items.pop(0)
22    def peek(self) -> Optional[T]:
23        """Return the front item without removing it, or None if empty."""
24        return None if self.is_empty() else self._items[0]
25    def is_empty(self) -> bool:
26        """Return True if the queue is empty."""
27        return len(self._items) == 0
28    def size(self) -> int:
29        """Return number of items in the queue."""
30        return len(self._items)
31    if __name__ == "__main__":
32        # Quick demo
33        q = Queue[int]()
34        print("Empty:", q.is_empty())
35        q.enqueue(10)
36        q.enqueue(20)
37        q.enqueue(30)
38        print("Peek/Front:", q.peek())
39        print("Size:", q.size())
40        print("Dequeue:", q.dequeue())
41        print("Dequeue:", q.dequeue())
42        print("Dequeue:", q.dequeue())
43        print("Empty:", q.is_empty())
```

OUTPUT:



The screenshot shows a PowerShell terminal window with the following output:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_11.1.py"
Empty: True
Peek/Front: 10
Size: 3
Dequeue: 10
Dequeue: 20
Dequeue: 30
Empty: True
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods. Sample Input Code:

class Node:

pass class

LinkedList:

pass

PROMPT: Generate the singly linked list with insert and display methods

```

1 from typing import Any, Iterable, Iterator, Optional
2 class Node:
3     __slots__ = ("value", "next")
4     def __init__(self, value: Any, next: Optional['Node'] = None) -> None:
5         self.value = value
6         self.next = next
7 class SinglyLinkedList:
8     """Simple singly linked list with insert (append) and display methods."""
9     def __init__(self, items: Optional[Iterable[Any]] = None) -> None:
10         self.head: Optional[Node] = None
11         self._size = 0
12         if items is not None:
13             for item in items:
14                 self.insert(item)
15     def insert(self, value: Any) -> None:
16         """Append 'value' to the end of the list."""
17         node = Node(value)
18         if self.head is None:
19             self.head = node
20         else:
21             cur = self.head
22             while cur.next is not None:
23                 cur = cur.next
24             cur.next = node
25             self._size += 1
26     def display(self) -> str:
27         """Return a string representation of the list values (e.g. 'a -> b -> c')."""
28         values = []
29         cur = self.head
30         while cur is not None:
31             values.append(repr(cur.value))
32             cur = cur.next
33         return " -> ".join(values)
34     def __iter__(self) -> Iterator[Any]:
35         cur = self.head
36         while cur is not None:
37             yield cur.value
38             cur = cur.next
39     def __len__(self) -> int:
40         return self._size
41 if __name__ == "__main__":
42     # Quick demo
43     lst = SinglyLinkedList()
44     print("Empty list display:", lst.display())
45     lst.insert(1)
46     lst.insert(2)
47     lst.insert(3)
48     print("After inserts:", lst.display())
49     print("Length:", len(lst))
50

```

OUTPUT:

```

PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/singly_linked_list.py"
Empty list display:
After inserts: 1 -> 2 -> 3
Length: 3
After inserts: 1 -> 2 -> 3
Length: 3

```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

PROMPT: Generate the Binary search tree with insert and traverse in-order method

```

ASS_11.1 U  AI ASS LAB_11.1.py U  singly_linked_list.py U  binary_search_tree.py U
binary_search_tree.py > BinarySearchTree > traverse_in_order
1  from typing import Any, Iterator, List, Optional
2  class Node:
3      __slots__ = ("value", "left", "right")
4      def __init__(self, value: Any) -> None:
5          self.value = value
6          self.left: Optional[Node] = None
7          self.right: Optional[Node] = None
8  class BinarySearchTree:
9      """A simple binary search tree with insert and in-order traversal."""
10     def __init__(self) -> None:
11         self.root: Optional[Node] = None
12     def insert(self, value: Any) -> None:
13         """Insert 'value' into the BST. Duplicates go to the right subtree."""
14         if self.root is None:
15             self.root = Node(value)
16             return
17         cur = self.root
18         while True:
19             if value < cur.value:
20                 if cur.left is None:
21                     cur.left = Node(value)
22                     return
23                 cur = cur.left
24             else:
25                 if cur.right is None:
26                     cur.right = Node(value)
27                     return
28                 cur = cur.right
29     def inorder(self) -> List[Any]:
30         """Return a list of values from an in-order traversal."""
31         out: List[Any] = []
32         def _inorder(node: Optional[Node]) -> None:
33             if node is None:
34                 return
35             _inorder(node.left)
36             out.append(node.value)
37             _inorder(node.right)
38         _inorder(self.root)
39         return out
40     def traverse_in_order(self) -> Iterator[Any]:
41         """Yield values in in-order using an iterative generator."""
42         stack: List[Node] = []
43         cur = self.root
44         while stack or cur is not None:
45             if cur is not None:
46                 stack.append(cur)
47                 cur = cur.left
48             cur = stack.pop()
49             yield cur.value
50             cur = cur.right
51 if __name__ == "__main__":
52     bst = BinarySearchTree()
53     for v in [7, 3, 9, 1, 5, 8, 10]:
54         bst.insert(v)
55     print("In-order (list):", bst.inorder())
56     print("In-order (generator):", list(bst.traverse_in_order()))
57

```

OUTPUT:

```

PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjt63/Desktop/AI ASS COD/binary_search_tree.py"
In-order (list): [1, 3, 5, 7, 8, 9, 10]
In-order (generator): [1, 3, 5, 7, 8, 9, 10]
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD>

```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

PROMPT: Generate Hash table with basic insert, search and delete methods

```
hash_table.py >
1 from typing import Any, List, Optional, Tuple
2 """A simple hash table using separate chaining.
3
4 Methods
5 - insert(key, value): store the value under key
6 - search(key): return the value for key or None
7 - delete(key): remove the key and return True if removed
8 """
9
10 @- [Explain | Add Comment] >
11 def __init__(self, capacity: int = 8) -> None:
12     if capacity <= 0:
13         raise ValueError("capacity must be > 0")
14     self._capacity = capacity
15     self._buckets: List[List[Tuple[Any, Any]]] = [[] for _ in range(capacity)]
16     self._size = 0
17
18 @- [Explain | Add Comment] >
19 def _bucket_index(self, key: Any) -> int:
20     return hash(key) % self._capacity
21
22 @- [Explain | Add Comment] >
23 def insert(self, key: Any, value: Any) -> None:
24     """Insert or update the key with the given value."""
25     idx = self._bucket_index(key)
26     bucket = self._buckets[idx]
27     for i, (k, _) in enumerate(bucket):
28         if k == key:
29             bucket[i] = (key, value)
30             return
31     bucket.append((key, value))
32     self._size += 1
33
34 @- [Explain | Add Comment] >
35 def search(self, key: Any) -> Optional[Any]:
36     """Return the value for key, or None if not found."""
37     idx = self._bucket_index(key)
38     bucket = self._buckets[idx]
39     for k, v in bucket:
40         if k == key:
41             return v
42     return None
43
44 @- [Explain | Add Comment] >
45 def delete(self, key: Any) -> bool:
46     """Delete key from the table. Return True if deleted, False if not found."""
47     idx = self._bucket_index(key)
48     bucket = self._buckets[idx]
49     for i, (k, _) in enumerate(bucket):
50         if k == key:
51             bucket.pop(i)
52             self._size -= 1
53             return True
54     return False
55
56 @- [Explain | Add Comment] >
57 def __len__(self) -> int:
58     return self._size
59
60 @- [Explain | Add Comment] >
61 def __repr__(self) -> str:
62     items = []
63     for bucket in self._buckets:
64         for k, v in bucket:
65             items.append(f"{k}: {v}")
66     return "(" + ", ".join(items) + ")"
67
68 if __name__ == "__main__":
69     # Quick demo
70     ht = HashTable()
71     ht.insert('apple', 3)
72     ht.insert('banana', 5)
73     ht.insert('orange', 2)
74     print('Table:', ht)
75     print("Search 'banana':", ht.search('banana'))
76     print("Search 'grape':", ht.search('grape'))
77     print("Delete 'orange':", ht.delete('orange'))
78     print('Table after delete:', ht)
79     print('Length:', len(ht))
80
```


OUTPUT:

```
td63/OneDrive/Desktop/AI ASS COD/hash_table.py
PS C:\Users\chunc_yhjd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjd63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjd63/OneDrive/Desktop/AI ASS COD/hash_table.py"
Table: {'apple': 3, 'banana': 5, 'orange': 2}
Search 'banana': 5
Search 'grape': None
Delete 'orange': True
Table after delete: {'apple': 3, 'banana': 5}
Length: 2
PS C:\Users\chunc_yhjd63\OneDrive\Desktop\AI ASS COD>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

PROMPT: Generate a graph using an adjacency list

```

1  class Graph:
2
3      - display()
4      - bfs(start)
5      - dfs_recursive(start)
6      - dfs_iterative(start)
7
8      @-> [Option] Add Comments: *
9
10     def __init__(self, directed: bool = False) -> None:
11         self.adj: Dict[Any, Set[Any]] = {}
12         self.directed = directed
13
14     @-> [Option] Add Comments: *
15     def add_vertex(self, v: Any) -> None:
16         if v not in self.adj:
17             self.adj[v] = set()
18
19     @-> [Option] Add Comments: *
20     def add_edge(self, u: Any, v: Any) -> None:
21         self.add_vertex(u)
22         self.add_vertex(v)
23         self.adj[u].add(v)
24         if not self.directed:
25             self.adj[v].add(u)
26
27     @-> [Option] Add Comments: *
28     def remove_edge(self, u: Any, v: Any) -> bool:
29         if u not in self.adj or v not in self.adj[u]:
30             return False
31         self.adj[u].remove(v)
32         if not self.directed:
33             self.adj[v].remove(u)
34         return True
35
36     @-> [Option] Add Comments: *
37     def neighbours(self, v: Any) -> Iterable[Any]:
38         return iter(self.adj.get(v, set()))
39
40     @-> [Option] Add Comments: *
41     def display(self) -> str:
42         lines: list[str] = []
43         for node in sorted(self.adj.keys(), key=lambda x: str(x)):
44             nbrs = sorted(self.adj[node], key=lambda x: str(x))
45             lines.append(f"{node}: {nbrs}")
46         return "\n".join(lines)
47
48     @-> [Option] Add Comments: *
49     def bfs(self, start: Any) -> list[Any]:
50         if start not in self.adj:
51             return []
52         visited: Set[Any] = set()
53         order: list[Any] = []
54         q: Deque[Any] = deque([start])
55         while q:
56             cur = q.popleft()
57             order.append(cur)
58             for n in sorted(self.adj[cur], key=lambda x: str(x)):
59                 if n not in visited:
60                     visited.add(n)
61                     q.append(n)
62         return order
63
64     @-> [Option] Add Comments: *
65     def dfs_recursive(self, start: Any) -> list[Any]:
66         order: list[Any] = []
67         visited: Set[Any] = set()
68
69         @-> [Option] Add Comments: *
70         def _dfs(u: Any) -> None:
71             visited.add(u)
72             order.append(u)
73             for n in sorted(self.adj.get(u, []), key=lambda x: str(x)):
74                 if n not in visited:
75                     _dfs(n)
76         if start in self.adj:
77             _dfs(start)
78         return order
79
80     @-> [Option] Add Comments: *
81     def dfs_iterative(self, start: Any) -> list[Any]:
82         if start not in self.adj:
83             return []
84         visited: Set[Any] = set()
85         stack: list[Any] = [start]
86         order: list[Any] = []
87         while stack:
88             cur = stack.pop()
89             if cur in visited:
90                 continue
91             visited.add(cur)
92             order.append(cur)
93             # push neighbours in reverse sorted order to visit smallest first
94             for n in sorted(self.adj.get(cur, []), key=lambda x: str(x), reverse=True):
95                 if n not in visited:
96                     stack.append(n)
97         return order
98
99 if __name__ == "__main__":
100     g = Graph(directed=False)
101     edges = [{"A", "B"}, {"A", "C"}, {"B", "D"}, {"C", "D"}, {"D", "E"}]
102     for u, v in edges:
103         g.add_edge(u, v)
104     print("Adjacency list:")
105     print(g.display())
106     print()
107     print("BFS from 'A':", g.bfs("A"))
108     print("DFS recursive from 'A':", g.dfs_recursive("A"))
109     print("DFS iterative from 'A':", g.dfs_iterative("A"))
110

```

OUTPUT:

```

PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python31
3\python.exe "c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/graphs.py"
Adjacency list:
A: ['B', 'C']
B: ['A', 'D']
C: ['A', 'D']
D: ['B', 'C', 'E']
E: ['D']

BFS from 'A': ['A', 'B', 'C', 'D', 'E']
DFS recursive from 'A': ['A', 'B', 'D', 'C', 'E']
DFS iterative from 'A': ['A', 'B', 'D', 'C', 'E']
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD>

```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's `heapq` PROMPT: Generate priority Queue using `py` module.

```

priority_queue.py > PriorityQueue > _repr_
1  import heapq
2  import itertools
3  from typing import Any, List, Optional, Tuple
4  class PriorityQueue:
5      """A simple priority queue using heapq.
6
7      Items are stored as (priority, tie_breaker, item). Lower 'priority' values
8      are popped first. 'tie_breaker' ensures a stable order for equal priorities.
9      """
10     def __init__(self) -> None:
11         self._heap: List[Tuple[int, int, Any]] = []
12         self._counter = itertools.count()
13     def push(self, item: Any, priority: int = 0) -> None:
14         """Push 'item' with numeric 'priority' (lower = higher priority)."""
15         heapq.heappush(self._heap, (priority, next(self._counter), item))
16     def pop(self) -> Any:
17         """Remove and return the highest-priority item. Raises IndexError if empty."""
18         if self.is_empty():
19             raise IndexError("pop from empty priority queue")
20         return heapq.heappop(self._heap)[2]
21     def peek(self) -> Optional[Any]:
22         """Return the highest-priority item without removing it, or None if empty."""
23         return None if self.is_empty() else self._heap[0][2]
24     def is_empty(self) -> bool:
25         """Return True if the queue has no items."""
26         return not self._heap
27     def __len__(self) -> int:
28         return len(self._heap)
29     def __repr__(self) -> str:
30         items = [f"(p={p}, item={i!r})" for p, _, i in sorted(self._heap)]
31         return f"PriorityQueue({', '.join(items) + '})"
32 if __name__ == "__main__":
33     # Demo: lower numeric value -> higher priority
34     pq = PriorityQueue()
35     pq.push('low', priority=5)
36     pq.push('medium', priority=3)
37     pq.push('urgent', priority=1)
38     pq.push('also medium', priority=3)
39
40     print('Peak:', pq.peek())
41     print('Size:', len(pq))
42     while not pq.is_empty():
43         print('Pop:', pq.pop())
44

```

OUTPUT:


```
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS COD/priority_queue.py"
Peak: urgent
Size: 4
Pop: urgent
Pop: medium
Pop: also medium
Pop: low
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

PROMPT: Generate a double-ended queue in deque

```

AI ASS LAB 11.1.py U singly_linked_list.py U binary_search_tree.py U priority_queue.py U
deque_ds.py > Deque > __repr__
1 from collections import deque
2 from typing import Any, Deque as DequeType, Iterable, List, Optional
3 class Deque:
4     """Double-ended queue wrapper around collections.deque."""
5
6     Methods:
7     ~ append_right(item), append_left(item)
8     ~ pop_right(), pop_left()
9     ~ peek_right(), peek_left()
10    ~ is_empty(), to_list(), clear()
11    """
12    def __init__(self, items: Optional[Iterable[Any]] = None) -> None:
13        self._dq: DequeType[Any] = deque(items) if items is not None else deque()
14    def append_right(self, item: Any) -> None:
15        self._dq.append(item)
16    def append_left(self, item: Any) -> None:
17        self._dq.appendleft(item)
18    def pop_right(self) -> Any:
19        if self.is_empty():
20            raise IndexError("pop_right from empty deque")
21        return self._dq.pop()
22    def pop_left(self) -> Any:
23        if self.is_empty():
24            raise IndexError("pop_left from empty deque")
25        return self._dq.popleft()
26    def peek_right(self) -> Optional[Any]:
27        return None if self.is_empty() else self._dq[-1]
28    def peek_left(self) -> Optional[Any]:
29        return None if self.is_empty() else self._dq[0]
30    def is_empty(self) -> bool:
31        return not self._dq
32    def to_list(self) -> List[Any]:
33        return list(self._dq)
34    def clear(self) -> None:
35        self._dq.clear()
36    def __len__(self) -> int:
37        return len(self._dq)
38    def __repr__(self) -> str:
39        return f"Deque({list(self._dq)})"
40    if __name__ == "__main__":
41        d = Deque()
42        print("Empty:", d.is_empty())
43        d.append_right(1)
44        d.append_right(2)
45        d.append_left(0)
46        print("After appends:", d)
47        print("Peek left:", d.peek_left())
48        print("Peek right:", d.peek_right())
49        print("Pop left:", d.pop_left())
50        print("Pop right:", d.pop_right())
51        print("Remaining:", d.to_list())
52

```

OUTPUT:

```

PS C:\Users\chunc_yh\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yh\OneDrive\Desktop\AI ASS COD\python.exe "c:/Users/chunc_yh/OneDrive/Desktop/AI ASS COD/deque_ds.py"
Empty: True
After appends: Deque([0, 1, 2])
Peek left: 0
Peek right: 2
Pop left: 0
Pop right: 2
Remaining: [1]
PS C:\Users\chunc_yh\OneDrive\Desktop\AI ASS COD>

```

Task Description #9 Real-Time Application Challenge – Choose the

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
 2. Event Registration System – Manage participants in events with quick search and removal.
 3. Library Book Borrowing – Keep track of available books and their due dates.
 4. Bus Scheduling System – Maintain bus routes and stop connections.
 5. Cafeteria Order Queue – Serve students in the order they arrive.
- Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority

Queue o Linked List o

Binary Search Tree (BST) o

Graph o Hash Table o

Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with

AI-assisted code generation

PROMPT:USING ABOVE GENERATE ONE CODE FOR IMPLEMENT

```

425_11.1.0 425_11.1.0.0 425_11.1.0.0 425_11.1.0.0 425_11.1.0.0 425_11.1.0.0
425_11.1.0 425_11.1.0.0 425_11.1.0.0 425_11.1.0.0 425_11.1.0.0 425_11.1.0.0

# Library due-date manager using a priority queue for next-due lookups.
#
# This module provides a simple 'LibraryManager' that tracks borrowed books
# and their due dates. The manager uses a min-heap (priority queue) keyed by
# due date so the next-due book can be retrieved in O(log n) time.
#
#
# From _future_ import annotations
# Import heapq
# Import heapq
# Import heapq
# From datetime import date, datetime
# From typing import Dict, Optional, Tuple
#
# @- [Optional] AddComment:
def _to_date(d) -> Date:
    if isinstance(d, date):
        return d
    if isinstance(d, str):
        # accept YYYY-MM-DD
        return datetime.strptime(d, '%Y-%m-%d').date()
    raise TypeError("due date must be a date or ISO date string")
#
# @- [Optional] AddComment:
class LibraryManager:
    """Manage borrowed books and provide next-due lookups.
    Internally uses:
    - a min-heap of (due_ord, the counter, book_id) for next-due queries
    - a dict mapping book_id -> (student_id, due_date) for fast lookup and returns
    The heap is lazily cleaned: returned books remain in the heap until popped.
    """
    #
    # @- [Optional] AddComment:
    def __init__(self) -> None:
        self._heap = [] # list[tuple(int, int, str)]
        self._counter = itertools.count()
        self._book_map: Dict[str, Tuple[str, date]] = {}
    #
    # @- [Optional] AddComment:
    def borrow_book(self, student_id: str, book_id: str, due_date) -> None:
        """Record that student_id borrowed book_id until due_date.
        due_date may be a datetime.date or an ISO date string 'YYYY-MM-DD'.
        Raises ValueError if the book_id is already borrowed.
        """
        due = _to_date(due_date)
        if book_id in self._book_map:
            raise ValueError("book (%s) already borrowed" % book_id)
        self._book_map[book_id] = (student_id, due)
        heapq.heappush(self._heap, (due.ordinal(), next(self._counter), book_id))
    #
    # @- [Optional] AddComment:
    def return_book(self, book_id: str) -> bool:
        """Return a book. Returns True if the book was recorded and removed.
        return self._book_map.pop(book_id, None) is not None
        """
    #
    # @- [Optional] AddComment:
    def clean_top(self) -> None:
        """Pop heap entries that are stale (book returned or moved)."""
        while self._heap:
            due_ord, _, book_id = self._heap[0]
            entry = self._book_map.get(book_id)
            if entry is None:
                heapq.heappop(self._heap)
                continue
            # real due = entry
            if real_due.ordinal() != due_ord:
                # due date changed (re-borrowed) so stale entry
                heapq.heappop(self._heap)
                continue
            break
    #
    # @- [Optional] AddComment:
    def next_due(self) -> Optional[Tuple[str, str, date]]:
        """Return (book_id, student_id, due_date) for the next-due book, or None.
        """
        self._clean_top()
        if not self._heap:
            return None
        due_ord, _, book_id = self._heap[0]
        student_id, due = self._book_map[book_id]
        return (book_id, student_id, due)
    #
    # @- [Optional] AddComment:
    def pop_next_due(self) -> Optional[Tuple[str, str, date]]:
        """Remove and return the next-due book, or None if none.
        """
        self._clean_top()
        if not self._heap:
            return None
        book_id = heapq.heappop(self._heap)
        student_id, due = self._book_map.pop(book_id)
        return (book_id, student_id, due)
    #
    # @- [Optional] AddComment:
    def list_all_due(self) -> list[tuple[str, str, date]]:
        """Return a list of all currently borrowed books sorted by due date.
        items = [(v[1], k, v[0]) for k, v in self._book_map.items()]
        # items are (due date, book id, student id)
        items_sorted = sorted(items, key=lambda t: t[0].ordinal())
        return [(book_id, student_id, due_date) for due_date, book_id, student_id in items_sorted]
    #
    # @- [Optional] AddComment:
    if __name__ == '__main__':
        mgr = LibraryManager()
        mgr.borrow_book('001', '0-100', '2010-01-01')
        mgr.borrow_book('002', '0-100', '2010-02-28')
        mgr.borrow_book('003', '0-100', '2010-01-01')
        print('Next due:', mgr.next_due())
        print('All due sorted:')
        for book_id, student_id, due in mgr.list_all_due():
            print(' ', book_id, student_id, due.isoformat())
        print('Pop next due -> ', mgr.pop_next_due())
        print('After pop, next due:', mgr.next_due())

```

Output:

```

PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS CDD>
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS CDD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS CDD/library_due_manager.py"
Next due: ('B-200', 's002', datetime.date(2026, 2, 28))
All due sorted:
- B-200 s002 2026-02-28
- B-100 s001 2026-03-01
- B-300 s003 2026-03-05
Pop next due -> ('B-200', 's002', datetime.date(2026, 2, 28))
After pop, next due: ('B-100', 's001', datetime.date(2026, 3, 1))
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS CDD>
After pop, next due: ('B-100', 's001', datetime.date(2026, 3, 1))
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS CDD>
After pop, next due: ('B-100', 's001', datetime.date(2026, 3, 1))
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS CDD>

```

Task Description #10: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

PROMPT: Select suitable data structures for the given Smart E-Commerce features and justify your choices in 2–3 sentences. Then implement any one feature as a Python program with comments and docstrings.


```
singly_linked_list.py u  binary_search_tree.py u  priority_queue.py u  AI ASS LAB_11.1.py u X
AI ASS LAB_11.1.py > ...
  Explain | Add Comment | X
1  class OrderQueue:
2      """
3      Order Processing System using Queue Data Structure.
4
5      This class manages customer orders in FIFO order.
6      The first order placed will be processed first.
7      """
8
9      Explain | Add Comment | X
10     def __init__(self):
11         """Initialize an empty order queue."""
12         self.orders = []
13
14     Explain | Add Comment | X
15     def place_order(self, order_id):
16         """
17         Add a new order to the queue.
18
19         Args:
20             order_id (int): Unique order ID
21         """
22         self.orders.append(order_id)
23         print(f"Order {order_id} placed.")
24
25     Explain | Add Comment | X
26     def process_order(self):
27         """
28         Process the first order in the queue.
29
30         Returns:
31             int: Processed order ID
32
33         Raises:
34             IndexError: If no orders exist.
35         """
36         if self.is_empty():
37             print("No orders to process.")
38             return None
39
40         order = self.orders.pop(0)
41         print(f"Order {order} processed.")
42         return order
43
44     Explain | Add Comment | X
45     def display_orders(self):
46         """Display all pending orders."""
47         print("Pending Orders:", self.orders)
48
49     Explain | Add Comment | X
50     def is_empty(self):
51         """
52         Check if queue is empty.
53
54         Returns:
55             bool: True if empty, False otherwise.
56         """
57         return len(self.orders) == 0
58
59
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI_ASS_COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI_ASS_COD\AI_ASS_LAB_11.1.py"
Order 101 placed.
Order 102 placed.
Order 103 placed.
Pending Orders: [101, 102, 103]
Order 101 processed.
Pending Orders: [102, 103]
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI_ASS_COD> █
```